

HSCTF 2020 Crypto - Unexpected

deuterium — 2020-06-06

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/HSCTF/crypto/Unexpected/2020-06-06-HSCTF-2020-unexpected.html>

Abstract

CTF writeup: Three RSA moduli borrow each other's primes in HSCTF 2020's Unexpected. Pairwise GCDs collect the debt and let us decrypt.

Unexpected

Challenge

284 Solves



Unexpected 100

rsa crypto beginner

Alice, Bob, and Carol are really close friends; in fact, they are so close, they even share the same primes in their RSA public keys! Alice has $N = PQ$, Bob has $N = QR$, and Carol has $N = PR$, where P, Q, R are 1024 bit primes. All three also use the same public exponent $e = 65537$. Can you recover the three plaintexts?

Author: AC



Unexpected.t...

Flag

Submit

We are given 3 N 's and 3 C 's and given that all three share primes i.e

```
N1 = p*q  
N2 = q*r  
N3 = r*p
```

This is a problem of classic RSA with a small twist.

Now, we know factoring N into their factors is hard. However this can be done easily considering we have a common factor between pairs of N i.e q is a common factor of $N1$

and N_2 , r is a common factor of N_2 and N_3 and p is a common factor of N_3 and N_1 .

Once we get p , q and r , we can easily calculate Euler Totien ϕ and the modular inverse of E with respect to respective ϕ s.

```
#python3
import gmpy2 #Cool asf library
N1 =
38957383022990595181291984223101696285305365571918905662109397816983723362574821865821
63630847612416277492034959243510457939210010336159061758606919109259916143600981918456
94219976273862479619083888950023878067522938346326780738415407413425107357217439202489
24864311254994469245730062087118108472723906195103958128561882475318159207975261025627
23333957594242603466996229335924848954210939152042149332307810693239925149256224795031
98275275233640187252001610614566747914409113016099887525686080909172127506919377373937
00573340419225199988132682785742608460838832642619205891147408234641923978509235459989
04365370408113

N2 =
30366839038196755057410911649454619471890049164946337663721762824094096949587012117482
7705049910151195696200383593275555293255586827283990400451317444723234406968971873530
09328159168983279864691581660934786104753412179240903083465924190464674345338750449624
67910816827412454823781492933993726545589296585820708539724548878546585457418005743439
3015528851718553553201220281739954820271979667081052363406511938025061398551356675540
35821244913278167483281279644337847638765972962358127443376905677516371878287187974727
63274584739701774515912518595304030321702159681013107390041635337676793942016114108329
74546802038041

N3 =
47934556772995491373822845850157500732391124143616805292559513182179603008413403990947
43130287927996565298160174555422185410320841942637374406558835150138631140265626020072
46465297338677272719254006205192965523555243914503610550143480198461212780882981014684
48694875291776426762455492993714874782804576738397254881958127445359284888447359505403
56920273038857127652414836352483913807655170699520816765863272825856765769043174406026
96406801725773808540096566197368155865465874787834217398459241108501824220103887738276
62394875645037284428213480647641660248510802586297514767656139975126202747592640762728
01682962144457

E = 65537

C1 =
39670847454612580435289475743668368845729102869504421732585392949117113693548719061351
32174792090663212136970669770059125223383374196043298648544199617235706250250895004596
12736934675744115710978556346050350466970024450696226499749911198313775828281699871502
98787319922606640366778813206033688280077061533219093984661087688138243010151221291524
75323198273042966108548020374750471195251107955335291618529515397701537614193876625270
94415537933400873451490021233979268224054475360645920086811082803271848565851436058022
79761088763528719053329398048019148262553185551141571625347918479950940376765392742423
2672209598509

C2 =
35500651375055155079893171335468349126306247387917665645225505184868349753466057698157
55188513512567023608236766095782592327636772926927433193452735590857245163507733193372
26043634439282120083618718026203533033564167432280901197175559735572797382863132012675
40487690891433594174639322140272778826035488177331948022022593928339832694084710663071
66293308177372513164743696402736322083477518666833633890167229698223457382474869425318
21199790024647950924227337611907877819668593060172268197128413003269501597578146759488
89452619359893315241689441429639604328313150295169366816755068743208048061924058540870
1379144341703
```

```

C3 =
92483527830768048096632861854526889507753255652541371608096042192598565449713032968815
62194859427369285625175528881639282708556594139589493015903020108626663310538383451965
18237383846281768395909801043955047640003147798786793258813501366000503338638933238548
60501616986568822829775078071024835932629569384566388705590790096753599988521790597200
61400962408313054846197969647136738392236320579054542139370543369625100515292663366297
30913756688411854427999570223208667606703681762027957427028839409594591627448224813082
07216977591633165506022144554619917166813605068647135771098934688503944100008376414202
1784018773006

q = gmpy2.gcd(N1, N2) # taking q as greatest common divisor of N1 and N2
r = gmpy2.gcd(N2, N3)
p = gmpy2.gcd(N3, N1)

phi1 = (p - 1) * (q - 1) # calculation of phi
phi2 = (q - 1) * (r - 1)
phi3 = (r - 1) * (p - 1)

d1 = gmpy2.invert(E, phi1) # taking inverse of E modulo phi1
d2 = gmpy2.invert(E, phi2)
d3 = gmpy2.invert(E, phi3)

m1 = pow(C1, d1, N1) # decrypting ciphertexts with corresponding keys
m2 = pow(C2, d2, N2)
m3 = pow(C3, d3, N3)

print(bytes.fromhex(hex(m1)[2:] + hex(m2)[2:] + hex(m3)[2:]).decode())
# flag{n0_0n3_3xp3ct5_th3_sp4nish_inquisiti0n!} I wasnt expecting that either

```

Interactive controls are omitted from this PDF. See the canonical HTML version.